

基于概率搜索的电子产品多工序优化决策与抽样检测

摘要

本文是对于电子产品零配件组装的研究，主要分为“**抽样检查**”与“**组装决策**”两个部分。对于前者，我们分别构造了两个精确度不同与抽样样本量相关的成本目标函数，通过最大化目标来寻找最佳抽样样本量。对于后者，我们分别在少工序系统中构建了精确的**深度优先搜索**来找精确最优解，在多工序系统中构建了**改进遗传算法**来逼近最优解。

对于问题一，我们首先应用 **Cochran 公式**建立起可接受误差 e 与采样样本量 n 的关系，可接受误差 e 与修正项**负熵函数**构建出误差成本，误差成本再与检测成本联立成微分方程。解得“在 95%...”的最佳抽样样本量是 **986(985.7319)** 次，“在 90%...”的最佳抽样样本量是 **5091(5091.147)** 次。

对于问题二，我们重点分析了企业在生产过程中的关键决策。我们采用**深度优先搜索 (DFS)** 算法，构建了一个包含生产决策点的**搜索树**，考虑了检测成本、次品损失、市场售价以及可能的调换损失等关键因素。在模型构建过程中，我们通过引入一种零件最多检测一次的**剪枝优化**，显著减少了搜索空间，提高了计算效率。通过搜索所有可能的生产路径，我们找到了利润最高的生产方案。通过对不同情景下的分析，模型结果显示当两种零件各买 1 个时，不同情况下的期望利润分别为 **29.9108、23.5141、23.0235、17.6900、25.8518、25.6825** 元，具体的决策方案见附录中的“q2/results.xlsx”。

对于问题三，由于问题规模的扩大，我们对模型二的算法进行了**复杂度分析**，并使用在时间效率上更优的**改进遗传算法**逼近最优解。我们采用**网格搜索**来确定启发式算法存在的超参数。训练过程见图 11，结果见表 1，收益可达 **112.0588** 倍，具体的决策方案见附录中的“q3/决策链.txt”。

对于问题四，我们首先根据第一问的结论找出随机变量分布的**方差与抽样次数**之间的关系，进而求出随机变量**概率密度**与抽样次数之间的关系。再构造一个仅由抽样次数决定的目标函数，通过遍历抽样次数，找出目标函数的最大值点，其对应的抽样次数即为所求。在第四问中，我们求出了第二题的第一个情况下的抽样次数为总的零件数量的 **19%**，期望收益为 **29.17**。

最后，我们对模型进行了**灵敏度分析**，在**总体和初始购入零件比例**的实验上符合预期，结果见图 14、图 15 证明模型拥有良好的鲁棒性与泛用性。

关键字： 抽样检查 组装决策 Cochran DFS 遗传算法

一、问题重述

1.1 问题背景

在电子产品的生产过程中，在市场售价维持稳定的情况下，厂家可以通过合理的生产布局来提高收益；但是，在实际生产中，企业面临着供应商欺诈、制造损失、不合格成品退还等问题。如何在有限信息的情况下完成抽样检测 → 零件装配 → 成品检测 → 次品处理的决策链，是维持电子企业正常运作并赚取利润的关键所在。

1.2 问题要求

问题 1 企业需要设计一个抽样检测方案，以尽可能少的检测次数来评估供应商提供的零配件（零配件 1 或零配件 2）是否符合标称的次品率（10%）。方案需考虑在 95% 的信度下判定零配件次品率超过标称值时拒收，以及在 90% 的信度下判定次品率不超过标称值时接收。

问题 2 企业需要在已知两种零配件和成品次品率的情况下生产过程中做出一系列决策。这些决策包括：是否对零配件 1 和零配件 2 进行检测以决定是否丢弃不合格的零配件、是否对装配好的成品进行检测以决定是否将不合格的成品丢弃或拆解、是否对用户购买的不合格品进行调换并承担相应的调换损失等。针对企业在生产中遇到的情况，制定具体的决策方案，并基于成本、次品率、检测成本和市场售价等指标，确定最优的生产策略。

问题 3 复杂生产流程的决策在复杂的生产场景下，企业的生产过程包含多个工序和更多零配件。每个工序和零件都伴随着不同的次品率、检测成本及装配成本。业需制定详细装配计划和装配策略，确保生产过程中次品率最小化、成本最优。

问题 4 得到在问题 2 和问题 3 中零配件、半成品和成品的次品率均是通过运用抽样检测方法得到的情况下，为问题 2 和问题 3 制定最优生产策略。

二、问题分析

2.1 问题一分析

对于问题一，题目要求我们在不同假设的情况下，选择最优抽样方案，来平衡抽样成本和由抽样不足带来的后续决策的机会成本，对此，我们采取了以下措施：

- **Cochran 公式**

Cochran 公式用于估算大样本量时的最小样本量，尤其适用于比例数据的抽样调查。在我们的模型中，我们选择 Cochran 公式来反映误差 E 与抽样样本量 n_0 的关系，以构建起平衡两种成本的“桥梁”。

- **负熵函数**

这个函数衡量的是一个二项随机变量的不确定性，具体来说，它是 Shannon 熵的一部分，用于描述二项分布下的离散随机变量的信息熵。

在我们的模型中，零件的次品率越接近 0.5，意味着后续决策的不确定性越高，系统的熵也随之增大，这将导致机会成本的上升。为此，我们引入了与负熵函数相关的惩罚项，以反映由于抽样不足所带来的决策失误。

2.2 问题二分析

对于问题二，我们面临着一个涉及零配件和成品次品率的企业生产决策问题。为了寻找最优的生产策略，我们采用深度优先搜索（DFS）算法，枚举所有可能的生产流程，并计算每种流程的潜在利润。通过比较这些利润，最终确定最佳的生产方案。由于时间复杂度较高，并且零件的次品率会随着成品的拆解不断变化，我们采取了以下措施：

- **贝叶斯公式**

贝叶斯公式是计算后验概率最常见的方式之一，我们使用贝叶斯公式来计算成品拆分前后的零件的次品率的变化

- **剪枝**

对于时间复杂度大的深度优先搜索算法，剪枝往往是加快运行效率的好方法。我们发

2.3 问题三分析

对于问题三，题目要求我们将决策方法推广到 m 个零部件、 n 道工序的更一般情形上。对此，我们采取了以下措施：

- **遗传算法**

通过对问题二 DFS 算法的复杂度分析，该确定性算法的时间耗费会随着 m 、 n 规模的增加而以不可接受的速度增长。所以，我们考虑使用启发式算法，其中遗传算法更适合解决此种离散决策问题，故选择在牺牲一定精度的前提下，将算法的时间控制在多项式复杂度之内。

2.4 问题四分析

对于问题四，题目要求我们自己检测次品率，因此之前确定的次品率变成了一个随机变量。我们需要权衡检测成本和精确的参数对决策的影响这两个命题。对此，我们构造了一个能够反映最终利润的目标函数，这个目标函数只由检测的次数决定。在对检测次数的枚举中，我们找出使得目标函数最大时对应的检测次数，这个检测次数就是我们要找的检测次数。

三、模型假设

为简化问题，本文做出以下假设：

- **零配件之间的装配是独立的，每个零配件对最终产品的影响是独立的**

假设零配件之间的装配是独立的有助于简化模型，因为它使得我们可以将每个零配件对成品质量的影响独立考虑。

- **成品与半成品拆卸的时候不会带来额外的零件损坏**

在实际生产过程中，拆卸过程可能会对零配件造成一定的损坏，但在模型中假设不会有额外损坏可以帮助简化决策和成本分析。

- **抽样与零配件组装都符合正态分布**

抽样与零配件组装为正态分布是展开后续概率计算的基础。

四、符号说明

此处给出了部分的符号声明，其余符号在使用前均会定义。

符号	说明	单位
D_{si}	零配件 i 的次品率	%
P_{si}	零配件 i 的购买单价	元/件
T_{si}	零配件 i 的检测成本	元/件
D_{mi}	半成品 i 的次品率	%
P_{mi}	半成品 i 的购买单价	元/件
T_{mi}	半成品 i 的检测成本	元/件
D_{fi}	成品 i 的次品率	%
P_{fi}	成品 i 的购买单价	元/件
T_{fi}	成品 i 的检测成本	元/件
p	销售商声称的次品率	%
n	抽样样本个数	个
e	可接受的误差范围	/

五、问题一的模型的建立和求解

对于问题一，我们应用 Cochran 公式建立起了误差 e 与抽样样本 n_0 的关系，在引入熵的基础上，分别使用微分方程建立机会成本和检测成本与抽样样本数、容许接受误差 e 的关系，解出两者交点作为平衡点。最终两种情形的最佳抽样次数分别为 **986(985.7319)** 次与 **5091(5091.147)** 次。具体流程如图 1 所示

5.1 理论基础

- **Cochran 公式**

Cochran 法用于确定样本量的大小，以便在进行调查或实验时能够得出准确且有代表性的结果。通过对样本误差的一定妥协与限制，来确定所需的最小样本量，以达到预期的精度水平，其具体形式如公式 1：

$$n = \frac{Z^2 p(1-p)}{e^2} \quad (1)$$

在适用于本题的二项分布情况下，可以简洁的推导出 Cochran 公式：

$$E(x) = np \quad (2)$$

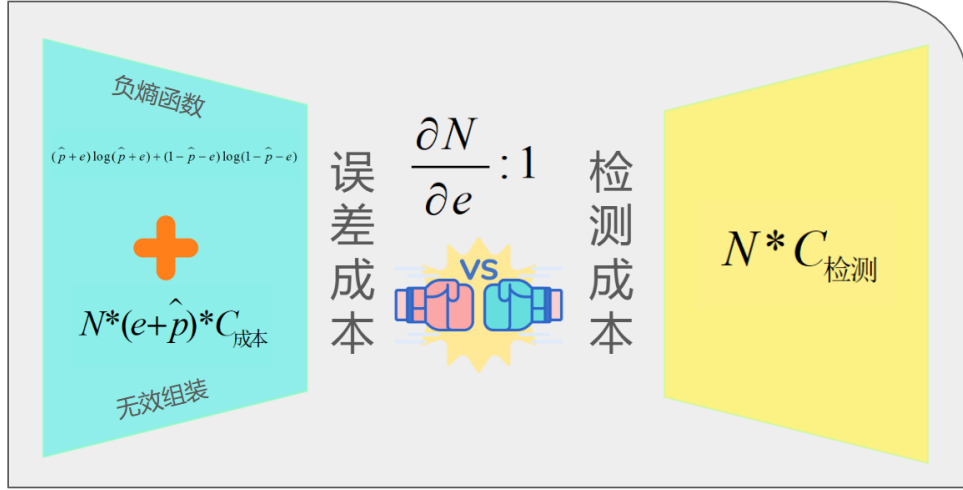


图1 问题一随采样样本 N 变化的两个成本及其关系

$$D(x) = np(1 - p) \quad (3)$$

公式 2、3 对应着二项分布的期望与方差，当 n 足够大时，根据中心极限定理，可以将其视为正态分布，故对与置信度为 α 假设检验，可列出公式 4：

$$P\left(-Z_{\alpha} \leq \frac{n(\hat{p} - p)}{\sqrt{np(1 - p)}} \leq Z_{\alpha}\right) = 1 - \alpha \quad (4)$$

不妨将误差设为 e：

$$e = Z \sqrt{\frac{p(1 - p)}{n}} \quad (5)$$

变形后即可得到 Cochran 公式的一般形式。

特别的是，当 n 不充分大的时候，需要引入修正项（公式 6）：

$$n = \frac{n_0}{1 + \frac{n_0 - 1}{N}} \quad (6)$$

• 负熵函数

熵定义为离散随机事件的出现概率，特别的，表示用以消除这个事件的不确定性所需要的统计信息量，即信息熵 H(x)：

$$H(x) = - \sum_{i=1}^n p_i \log_2 p_i \quad (7)$$

对于次品挑选问题（二项分布），公式 7 可以退化为：

$$H(x) = -p \log_2 p - (1 - p) \log_2 p \quad (8)$$

5.2 模型求解

由于在现实情况中不存在总体无限大的情况，故在本例中不妨将总体 N 设置为 1000000，其组装成本参考题目给出表一的数值：

对于信度不同、原假设不同的情形，模型求解方式相似，故先以“在 95% 的信度下认定零配件次品率超过标称值，则拒收这批零配件；”为例阐述模型，在给出其他情形的解答：

5.2.1 步骤 1：计算抽样样本量 n 与误差范围 e 的函数关系：

对于“在 95% 的信度下认定零配件次品率超过标称值，则拒收这批零配件；”的情形，那么接受这批零件的概率为：

$$P\left(\frac{n(\hat{p} - p)}{\sqrt{np(1-p)}} \leq -Z_\alpha\right) = 1 - \alpha \quad (9)$$

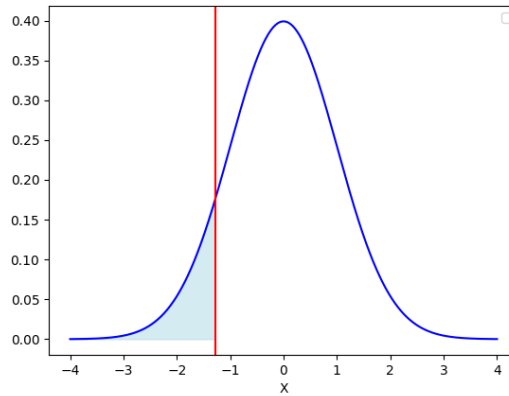


图 2 假设检验图

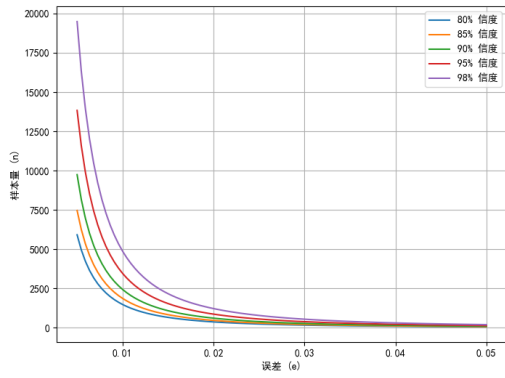
由 Cochran 公式可得公式 10：

$$\begin{cases} n_0 = \frac{Z_\alpha^2 p(1-p)}{E^2} \\ n = \frac{n_0}{1 + \frac{n_0 - 1}{N}} \end{cases} \quad (10)$$

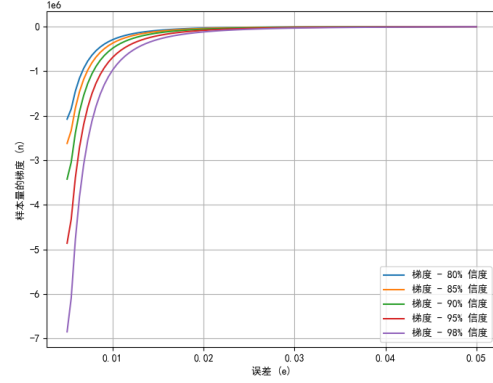
联立易得：

$$n = \frac{Z_\alpha^2 N p(1-p)}{e^2 N + Z^2 p(1-p)} \quad (11)$$

对于不同的信度，我们可以得到以下几条不同的 n - e 曲线以及其梯度图，如图 3：可以看到，抽样样本越少，对 e 的限制越宽。



(a) n-e 关系图



(b) n-e 梯度图

图 3

5.2.2 步骤 2：计算误差成本与检测成本的平衡曲线：

在任意一个时间节点，企业是否需要继续进行抽样只取决于此时抽样的边际收益是否为正，对此，我们定义误差成本为：

$$Cost_{\text{误差}} = N * (e + \hat{p}) * C_{\text{成本}} + N * \text{loss}(e) \quad (12)$$

其中： $N * (e + \hat{p}) * C_{\text{成本}}$ 代表组装错误零配件带来的成本损失， $N * \text{loss}(e)$ 代表由于次品率升高带来的决策成本，这里的 e 取负数，原因是其上界为 $-e + \hat{p}$ ，若为其他情形，则可能不同。

同时，综合图 3 (b) 给出的梯度，我们可以得到：

$$\begin{cases} \frac{\partial n}{\partial e} * C_j = N * (e + \hat{p}) * C_{jb} + N * \text{loss}(e) \\ \text{loss}(e) = (\hat{p} + e) \log(\hat{p} + e) + (1 - \hat{p} - e) \log(1 - \hat{p} - e) \end{cases} \quad (13)$$

loss 采用**负熵函数**来代表决策困难的惩罚，该微分方程组可解得如下函数图：其解析式近似为：

$$y = 1.775e + 07x - 3.353e + 06x + 3.851e + 05x + 1.437e + 05x + 3938x - 54.1 \quad (14)$$

5.2.3 步骤 3：寻找符合信度与假设的成本平衡点：

经过前两个步骤，我们已经找出来成本平衡曲线与 n-e 关系图，下面我们将他们合并到一张图上，如图 5：在平衡点上，需要抽样 **986(985.7319)** 次，误差 e 为 **3.75%**。

5.2.4 情形 2 的求解：

情形 2 为：“在 90% 的信度下认定零配件次品率不超过标称值，则接收这批零配件。”与情景 1 唯一不同的是这里的 e 变成了正数，由模型可以得到以下结果：

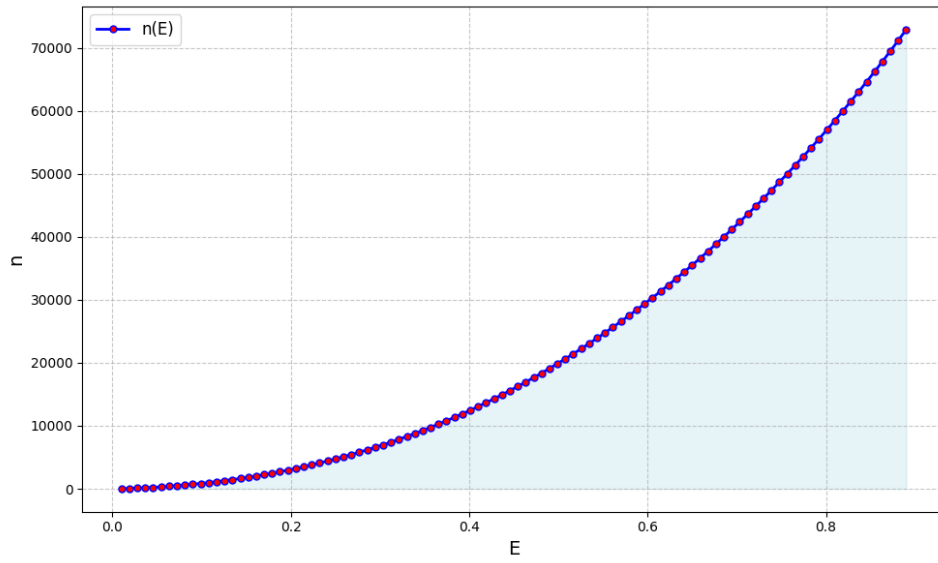


图4 成本平衡曲线

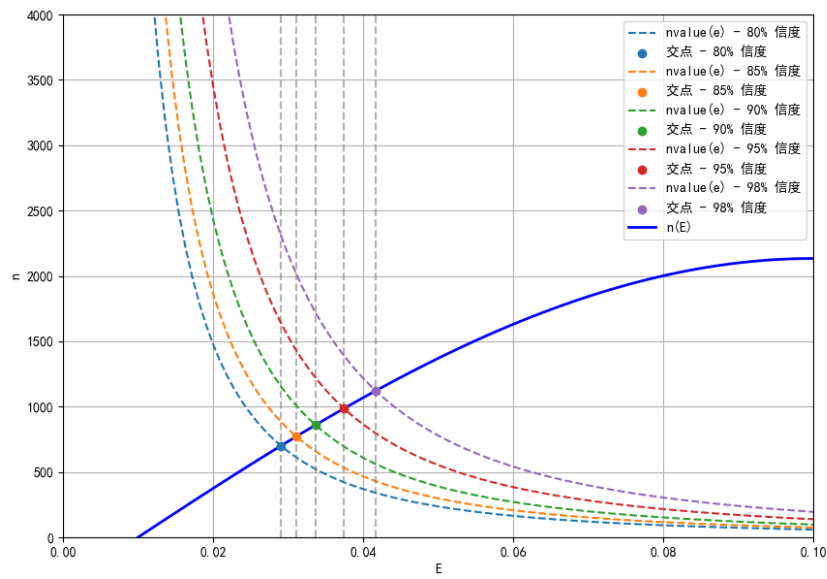


图5 情况一平衡点

平衡曲线解析式为

$$y = 2.447e + 04x - 4.845e + 04x + 7.035e + 04x + 4.635e + 04x + 3796x - 45.68 \quad (15)$$

在平衡点上，需要抽样 **5091(5091.147)** 次，误差 e 为 **7.35%**。

5.3 结果分析

综上所述，情形 1 的最佳抽样次数为 **986(985.7319)** 次，误差 e 为 **3.75%**；

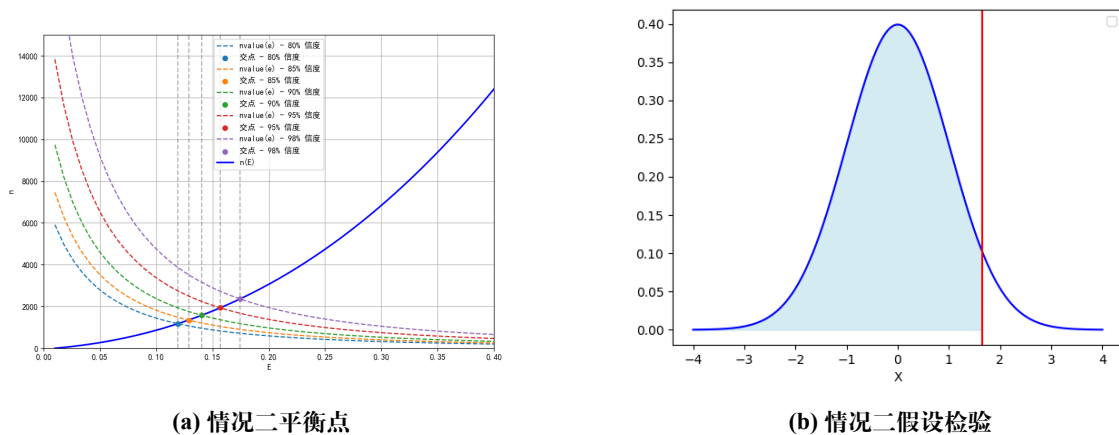


图 6

情形 2 的最佳抽样次数为 **5091(5091.147)** 次，误差 e 为 **7.35%**；

六、问题二的模型的建立和求解

对于问题二，我们采纳**深度优先搜索**策略来遍历潜在的生产流程方案，并通过**剪枝**技术进行高效优化。

6.1 理论基础

• 深度优先搜索

递归结构：树是通过递归调用来构建的，每个递归调用都会创建一个新的树节点。

路径表示：树中的每一条路径代表了一种可能的操作序列，这些操作是在解决问题时可以采取的。

回溯机制：在 DFS 中，当到达一个节点且没有更多的选择时，算法会回溯到上一个节点，尝试其他未探索的路径。

剪枝：代码中包含了一些条件检查，这些检查可以提前终止某些路径的搜索，从而优化算法的性能

• 贝叶斯公式

贝叶斯公式是一种计算后验概率的方式。其基本的思想是：通过新的证据“B”，可以更新对事件‘A’的相信程度

贝叶斯公式的基本形式如下：

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)} \quad (16)$$

其中， $P(A|B)$ 表示已知事件 B 发生的情况下，事件 A 发生的概率， $P(B|A)$ 表示已知事件 A 发生的情况下，事件 B 发生的概率， $P(A)$ 和 $P(B)$ 分别代表 A,B 发生的概率

在本题中，我们使用贝叶斯公式，来计算将成品拆分之后零件的损坏率是多少。

6.2 模型求解

6.2.1 构建搜索树

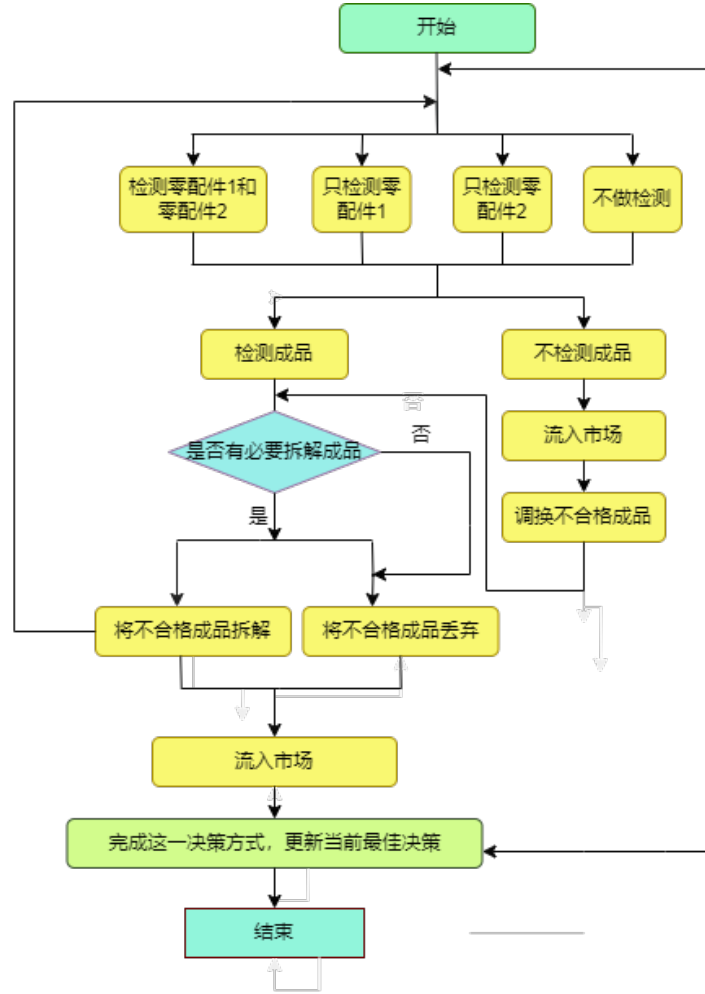


图7 企业生产决策流程图

我们构建了一个搜索树，每个节点代表一个生产决策点。树的深度反映了生产过程的不同阶段，包括零配件检测、成品检测、产品拆解和调换等。每个节点的状态由零配件和成品的次品率、当前的成本和盈利状况等关键参数定义。

在做选择的过程中，我们发现，零配件1的次品率 D_{s1} 、零配件1的数量 N_{s1} 、零配件2的次品率 D_{s2} 、零配件2的数量 N_{s2} 、成品的次品率 D_f 、成品的数量 N_f 、当前已花费的成本 $Cost$ 和当前已得到的收入 $Income$ 以及当前搜索所处的阶段 $step$ ， $step$ 的值取1、2、3、4分别表示零配件检查、成品检测、不合格成品拆解和投入市场阶段。。这些变量会随着选择的不同发生变化，同时这些变量的不同也会影响着我们所做出的决策。如果我们用 $step$ 表示当前搜索所处阶段我们把这些变量列成一个向量如下

$$state : (D_{s1}, N_{s1}, D_{s2}, N_{s2}, D_f, N_f, Cost, Income, step) \quad (17)$$

我们将这个向量称之为节点的状态，每个节点有其唯一的状态。

每个节点有若干个不同的选择，例如 $\text{step}=1$ 时，我们可以选择同时检测零配件 1 和零配件 2、只检测零配件 1、只检测零配件 2 和不做任何检测这四个不同方式。我们分别做出这四个选择后会产生四个新的节点以及对应的状态，这些节点就成为了之前节点的“儿子”。

下面的表列出了不同 step 的不同选择，以及对应的下一个节点的 step 。

step	选择	下一个 step
1. 零配件检查	1. 同时检测零配件 1 和零配件 2	2. 成品检测
1. 零配件检查	2. 只检测零配件 1	2. 成品检测
1. 零配件检查	3. 只检测零配件 2	2. 成品检测
1. 零配件检查	4. 不做任何检测	2. 成品检测
2. 成品检测	1. 检测	4. 投入市场
2. 成品检测	2. 不做检测	3. 不合格成品拆解
3. 不合格成品拆解	1. 不拆解	结束
3. 不合格成品拆解	2. 拆解	1 零配件检查
4. 投入市场	1. 不合格成品调换	3. 不合格成品拆解

利用上面的表，我们可以不断产生新的节点，做出不同的选择。

6.2.2 边界条件

我们在搜索的过程中不断在搜索树上产生新的节点，但是我们不可能无休止地创建新的节点，必须找出**边界条件**来终止搜索。

不难发现，产生循环的原因主要在 $\text{step}3$ 中的不合格成品拆解。拆解完的不合格成品将会变成两个零件，而两个零件又会产生新的成品，成品中又会有不合格的成品出现，这样就产生了循环。那么什么时候终止对不合格成品的拆解，成为了我们确定边界条件的关键。

我们考虑最后一轮不拆解的情况。假设这轮当中零配件 1 的次品率为 p_1 ，零配件 1 的数量为 n_1 ，零配件 2 的次品率为 p_2 ，零配件 2 的数量为 n_2 ，那么这一轮合成的成品数量应为 $\min(n_1, n_2)$ ，则总的装配成本应为 $\min(n_1, n_2) * \text{Cost}_{\text{装配}}$ ，其中 $\text{Cost}_{\text{装配}}$ 表示 1 个成品的装配成本。而合格的产品数量应为

$$n_{\text{合格}} = \min(n_1, n_2) * (1 - p_1) * (1 - p_2) * (1 - p_{\text{合成}}) \quad (18)$$

将成品合成之后，我们可以选择检测后卖出或者是直接卖出。检测后卖出的成本应为 $\min(n_1, n_2) * \text{Cost}_{\text{检测}}$ 。直接卖出的期望成本应为 $\min(n_1, n_2) * (1 - (1 - p_1) * (1 - p_2) * (1 - p_{\text{合成}})) * \text{Cost}_{\text{调换}}$ ，其中 $\text{Cost}_{\text{调换}}$ 代表 1 个失败成品的调换费用。我们可以从这两个成本中选择较小的那一个作为本轮决策的成本即

$$\text{cost} = \min(\min(n_1, n_2) * \text{Cost}_{\text{检测}}, \min(n_1, n_2) * (1 - (1 - p_1) * (1 - p_2) * (1 - p_{\text{合成}})) * \text{Cost}_{\text{调换}}) \quad (19)$$

全部卖出后的利润 $lr = n_{\text{合格}} * \text{value}$ 其中 value 代表市场售价。

因此在此轮中，如果成本大于了利润，那么在上一轮中就不应该选择拆解损坏的零件，而应该直接丢弃，即： $lr < cost$ 化简得：

$$(1-p_1)*(1-p_2)*(1-p_{\text{合成}})*value > Cost_{\text{检测}} + Cost_{\text{装配}}(1-(1-p_1)*(1-p_2)*(1-p_{\text{合成}}))*Cost_{\text{调换}} \quad (20)$$

不难发现，这个不等式只与 p_1 p_2 有关，所以我们计算了处于边界条件的 p_1 与 p_2 可能的情况，如下图。

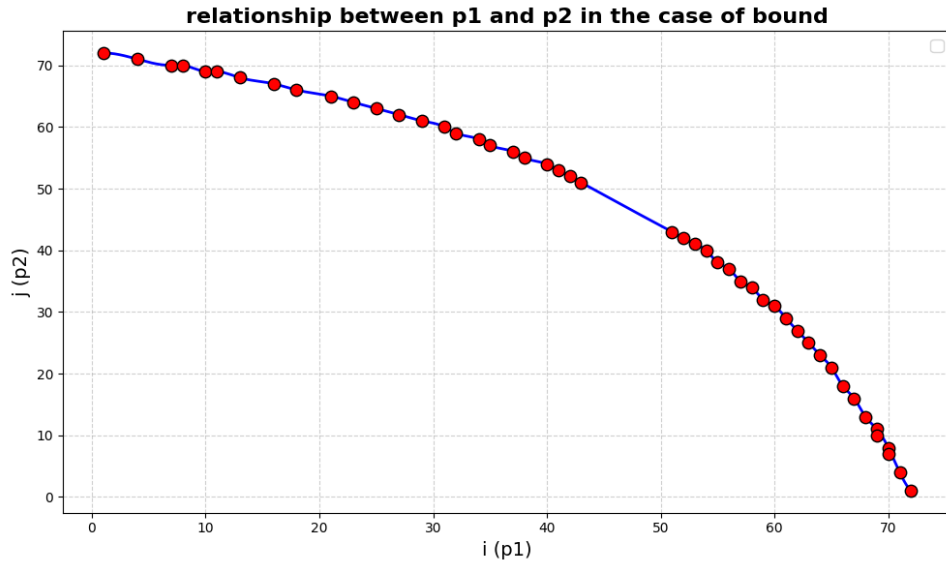


图8 边界条件下 p_1 与 p_2 的关系

6.2.3 利润计算模型

对于决策树上的每一个节点搜索的过程中，我们根据选择的不同更新子节点的状态。大部分状态的变化较为简单，例如当我选择对一个零件检测的时候 (step1, 选择 2)，新的状态的零件 1 的数量 $N'_{a1} = N_{a1} * D_{a1}$ ，而零件 1 的次品率，由于检测之后我们必然会将损坏的零件丢掉，故次品率 $D'_{a1} = 0$ 。简单的变化再次不再赘述，我们详细说说对于不合格成品拆分成零件这一过程。

假设不合格成品的数量为 N ，本轮中合成成品的零件 1 的数量和次品率分别为 n_1, p_1 ，零件 2 的数量和次品率分别为 n_2, p_2 ，则合成零件中，不合格零件的数量：

$$N = \min(n_1, n_2) * (1 - (1 - p_1) * (1 - p_2) * (1 - p_{\text{合成}})) \quad (21)$$

我们令命题 A 为零件 1 不合格的情况，命题 B 为零件 2 不合格的情况，命题 C 为成品 3 不合格的情况，则我们拆分的不合格成品中，零件 1 不合格的概率为：

$$P(A|C) = \frac{P(C|A) * P(A)}{P(C)} \quad (22)$$

由于当一个零件损坏时，成品一定损坏，故 $P(C|A)=1$ ，则

$$P(A|C) = \frac{P(C|A) * P(A)}{P(C)} = \frac{P(A)}{P(C)} = \frac{p_1}{1 - (1 - p_1) * (1 - p_2) * (1 - p_{\text{合成}})} \quad (23)$$

则拆分后零件 1 的不合格率

$$p'_1 = \frac{(n_1 - \min(n_1, n_2)) * p_1 + N * P(A|C)}{n_1 - \min(n_1, n_2) + N} = \frac{n_1 * p_1}{n_1 - \min(n_1, n_2) + N} \quad (24)$$

同理，零件 2 的不合格率

$$p'_2 = \frac{n_2 * p_2}{n_2 - \min(n_1, n_2) + N} \quad (25)$$

6.2.4 剪枝

对于 step1，我们可以选择是否对两个零件进行检测。值得注意的是，如果我已经对一个零件进行过检测，并将检测出的不合格零件丢弃。此时我可以确定这种零件不会存在不合格品，即次品率 $D'_{a1} = 0$ 。在知道所有零件都合格后，再进行检测将毫无意义。因此，在整个搜索过程中，我们只需对一个零件进行**最多一次检测**即可。

同时，我们注意到，如果两个零件都已经进行了检测，那么在 step1 中，我们将不需要对两个零件进行检测。那么循环中的决策就只剩下了两条：成品是否检测以及是否拆解。

实际上，由于我们可以确保零件都是完好的，那么一次循环的收益与剩余零件数量的比值应该是固定值，并且只与合成成品的次品率有关。

假设合成成品的次品率为 $p_{\text{合成}}$ ，零件 1、2 的数量分别为 n_1, n_2 则
装配成品的成本为：

$$Cost_0 = \min(n_1, n_2) * cost_{\text{装配}} \quad (26)$$

一次检测的成本为：

$$Cost_1 = \min(n_1, n_2) * cost_{\text{检测}} \quad (27)$$

若不检测，直接出售，需要支付的调换费用为：

$$Cost_2 = \min(n_1, n_2) * p_{\text{合成}} * cost_{\text{调换}} \quad (28)$$

而出售获得的收入为

$$Income = \min(n_1, n_2) * 1 - p_{\text{合成}} * value \quad (29)$$

则总利润为：

$$lr = Income - Cost_0 - \min(Cost_1, Cost_2) \quad (30)$$

不难发现，当 $Cost_1 > Cost_2$ ，即 $cost_{\text{检测}} > p_{\text{合成}} * cost_{\text{调换}}$ 时，选择直接出售，否则选择先检测后出售。当利润 $lr > 0$ 时，一直拆分不合格品，直到所有成品都合格为止。

否则直接丢弃不合格的成品。

考虑一直拆分不合格品的情况，由于最终所有成品都合格，所以我们只需计算成本即可。我们不妨设 $n_1 < n_2$ ，则第一轮执行时，成品的数量应为 $N_1 = n_1$ ，我们令 $Cost = \min(Cost_0, Cost_1)$ ，则第一轮执行的成本应为 $C_1 = Cost$ ，剩余不合格品的数量应为 $N_2 = N_1 * p_{\text{合成}}$ ，这也是第二轮循环装配出来的成品的数量。由于成本应该与数量呈正比例关系，故

$$\frac{C_2}{C_1} = \frac{N_2}{N_1} \quad (31)$$

解得 $C_2 = p_{\text{合成}} * Cost$ 。

以此类推，第 n 轮循环的成本应为

$$C_n = p_{\text{合成}}^{n-1} * Cost \quad (32)$$

则总成本 C 应为

$$C = \sum_{i=1}^{\infty} C_i = \sum_{i=1}^{\infty} p_{\text{合成}}^{i-1} * Cost = \frac{Cost}{1-p} \quad (33)$$

总利润为

$$LR = n_1 * value - \frac{Cost}{1-p} \quad (34)$$

使用该方法，可以直接获得当两个零件都被检测过后的选择方式和利润，以此可以加快代码运行的效率。

6.3 结果分析

指标结果包括每种决策方案的利润，以及在不同阶段采取的生产决策等参数。通过比较这些指标，我们可以确定最佳的生产策略，找到的最优决策链详见附件“result/q2.xls”，。

针对情况 1，我们的模型得出其最大期望利润为 29.9108；

针对情况 2，我们的模型得出其最大期望利润为 23.5141；

针对情况 3，我们的模型得出其最大期望利润为 23.0235；

针对情况 4，我们的模型得出其最大期望利润为 17.6900；

针对情况 5，我们的模型得出其最大期望利润为 25.8518；

针对情况 6，我们的模型得出其最大期望利润为 25.6825；

在深入探索了所有潜在情况后，我们成功识别并实施了利润最大化的策略，确保了结果的准确性和可靠性。

七、问题三的模型的建立和求解

对于问题三，我们进一步分析了更复杂的模型，将决策链抽象为离散的“基因”，使用启发式算法（遗传算法）求解，最终对于题目给出的情景，找到的最优决策链详见附件

件”q3/决策链.txt”，最优收益成本比为：**112.0588**，算法伪代码如下：

Algorithm 1 基于改进遗传算法的零件装配伪代码

```
1: 初始化 ( $D_{si}$ 、 $P_{si}$ 、 $T_{si}$ 、 $D_{mi}$ 、 $P_{mi}$ 、 $T_{mi}$  ... ..):
2: 设置参数 (pop、max_iter、initial_pop、mutation_prob ... ..)
3: 生成初始种群
4: for 每一代  $g$  从 1 到  $max\_iter$  do
5:     计算每个个体的适应度
6:     for 每个决策变量  $j$  从 1 到  $rd$  do
7:         计算费用
8:         if 零配件  $i$  被检测 || 半成品  $i$  被检测或拆解 || 成品被检测 then
9:             计算零配件/半成品/成品的价值损失
10:        end if
11:        if 决定丢弃成品 then
12:            Break
13:        end if 计算成品的拆解费用
14:    end for
15:    选择操作、交叉操作、变异操作
16:    更新种群、记录当前最佳适应度
17: end for
18: 输出最佳解和最佳适应度
```

7.1 理论基础

• 遗传算法：

遗传算法是模仿生物进化择优的启发式算法，主要通过交叉、变异、选择等操作来优化问题的解，能在牺牲一定精度的前提下，大幅度降低算法的复杂度。

• 复杂度分析：

m 个零件， n 层工序的电子加工问题与第二问的电子加工问题存在很大不同。从算法的时间复杂度来分析：

由于在 DFS 中，算法会遍历所有可能的路径，而每个路径上每个节点都会有多个分支，在考虑零配件只需检查一次的剪枝情况下，算法的时间复杂度依然是指数级的，准确来说，是 $O(3^n)$ ，这在规模较大的问题中是不可接受的；

而以遗传算法为代表的启发式算法，虽然不能保证找到正确的解，但是其循环的次数是有限的，经过一定设计，从伪代码可以得知，使用遗传算法解决电子零件配对问题的复杂度是 $O(n^2)$ ，增长速度如图 9，可见当 m 、 n 很大时，本模型也能正常求解：

7.2 模型求解

对于这个扩展的零件装配问题，我们选择以题目给出的 8 个零件，3 道工序作为特例，以解释遗传算法的整体流程：

其中，基因组由一系列决策组成，前 8 个分别是零配件检不检测（检测 1，不检测 0），9 到 11 是半成品的检测、拆解和不检测（检测丢弃 2，检测拆解 1，不检测 0），12

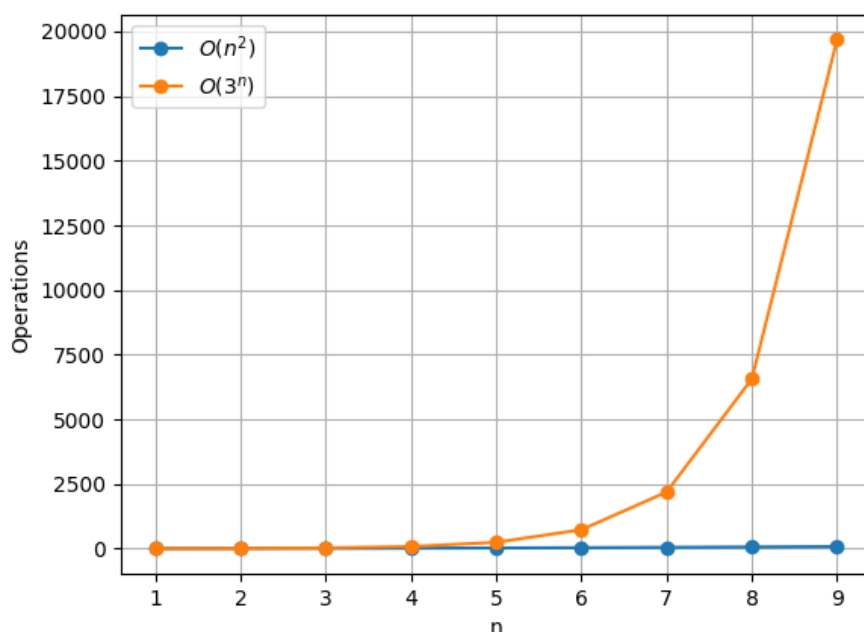


图9 时间复杂度增长情况

是成品是否检测（检测 1，不检测 0），13 是全部丢弃还是拆解（1 丢弃，0 拆解）

7.2.1 步骤 1：模型简化

为了让模型更快的收敛，我们对模型做出了一些额外的假设进行简化：

假设成品不能拆成半成品

每个零件的购入量都是一样的

半成品是拆卸还是丢弃取决于超参数 α ，当半成品的次品率超过 α 时则丢弃，否则拆卸

并不是所有的假设都是符合实际的，我们将在灵敏度分析中进一步阐述。

7.2.2 步骤 2：个体适应度计算

整个决策流程可化为 $\varepsilon - NFA$ ，如图/所示：其中：

- 零配件阶段 (q_i)

有 8 个决策点，分别代表着 8 个零部件是否检验，如检验则支付相应的检测费用，但是可以减少零件总数并提高良品率。

- 半成品阶段 (q_i)

有 3 个决策点，分别代表 3 种半成品是不检验 (0)，检验不丢弃 (1) 还是检验丢弃 (2)，如是 0，1 则正常进入下一阶段，如选择 2，因为此时该半成品极有可能被消耗完，故选择在完成组转后直接退出而不返回零配件阶段，将其剪枝，有效解决了传统遗传算法基因长度固定的问题。

- 成品阶段 (q_i)

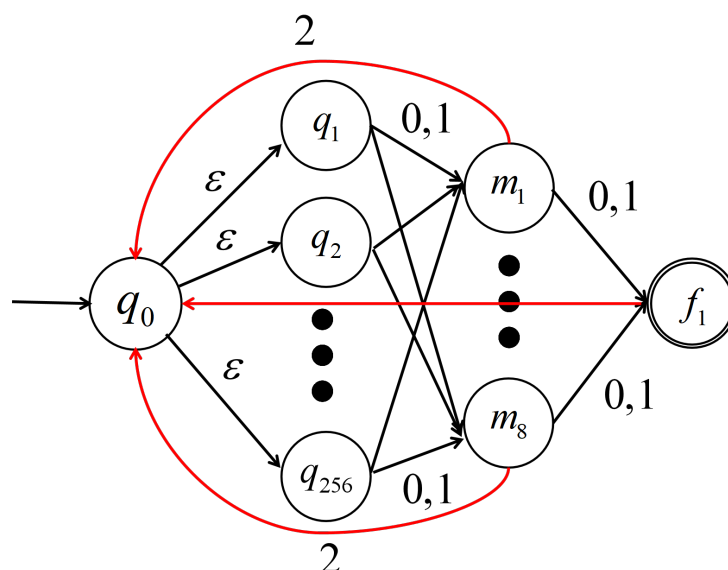


图 10 决策流程

有 1 个决策点，代表成品是否需要检验，但是从问题二的分析中可以得知，这是一个伪决策问题，可以将不检测产品而直接投入市场看作一种“检测”，只需要对比两者的耗费，就可以确定当前情况下的决策方案了。

- 丢弃阶段 (q_i)

有 1 个决策点，代表不合格的成品，我们可以选择拆卸或丢弃。

7.2.3 步骤 3：交换、变异与选择

对于变异方法，我们采用了交叉变异与突变两种：

- 交叉变异

从一个个体的基因组中挑出至多 20 个基因，将题目与邻家交换。

此处，我们将交叉变异的超参数 `crossover_prob` 设置为较高的 **0.6**，因为希望从相邻的个体上学到更优的排列方式。

- 交叉变异

从一个个体的基因组中挑出至多 50 个基因，改变他们。

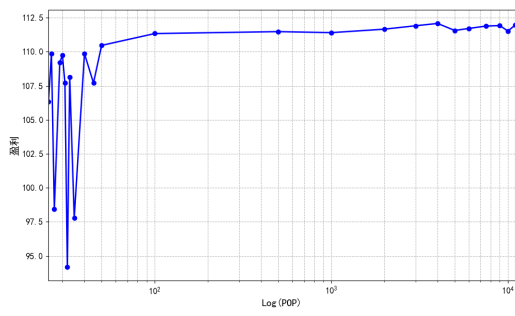
`mutation_prob` 设置为较低的 0.2，因为希望保留更有利的基因特征。

7.2.4 步骤 4：超参数选择

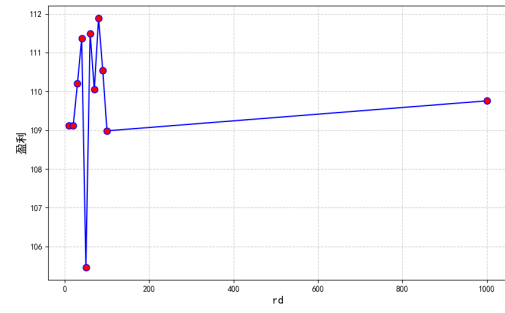
对于遗传算法，存在很多超参数可以调整，合适的参数可能带来模型能力的显著提升，我们使用**网格搜索**的方式来调整超参数 `rd`（预期收敛层数）和 `pop`（种群数量），在综合考虑时间与模型能力的情况下，选择 **`rd=20,pop=10000`**，如图 11 所示：

7.3 求解结果

综上所述，遗传算法得到的最优决策链见附件“**result q3**”，最优收益成本比为：**112.0588**。收敛过程见图/：经过对比，本算法显著的优于随机



(a) POP 网格搜索



(b) rd 网格搜索

图 11 超参数网格搜索

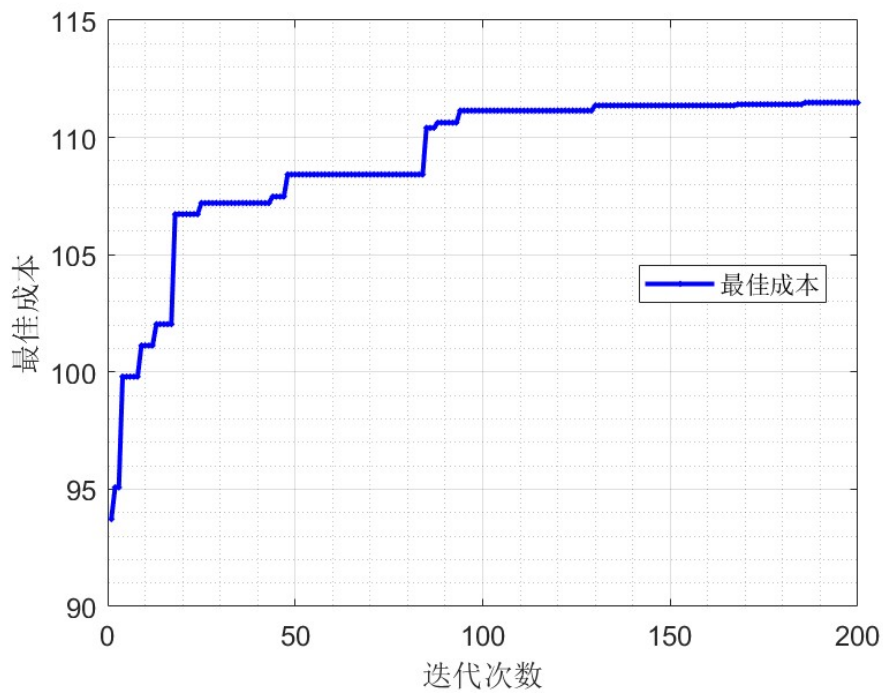


图 12 改进遗传算法收敛

选择与贪心算法，见表/。

	遗传算法	随机算法	贪心算法
盈利	112.0	3.3	14.2

表 1 算法对比

八、问题四的模型的建立和求解

8.1 模型建立

对于第四问，由于需要抽样检测来确定具体的分布，此时零件的次品率，合成的次品率都由先前的某一个数变成了一个随机变量。因此我们先构建了一个方差与抽样数量之间的关系，抽样数量越大，方差越小，再利用方差计算出变量分布的概率密度，再求出利润的期望，枚举抽样的数量，当满足利润期望最大时所对应的抽样的数量即为所求。

8.2 模型求解

Step1: 构建方差与抽样数量之间的关系

在第一问中，我们已经求出：

$$E = \sqrt{\frac{Z^2 p * (1 - p)}{n}} \quad (35)$$

我们不妨设 E 是正态分布最高点到右侧 $\frac{1}{2}$ 高度的点的水平距离，那么有

$$e^{-\frac{E}{2\sigma^2}} = \frac{1}{2} \quad (36)$$

解得：

$$\sigma = \sqrt{2 * \ln 2 * E} \quad (37)$$

则 p 的真实分布 $\hat{p} \sim N(p, \sqrt{2 * \ln 2 * E(n)})$

Step2: 计算利润的期望

第四问与二三问最大的区别在于，由于给定的 P 值不一定精准，真实的 $\hat{p}$ 可能会分布在 P 的两侧。因此我们需要先将给定的值附近的最大利润计算出来。考虑一个一维问题，我需要计算出分布在 P 附近的点的最大利润。其中 p 代表零件的次品率。记点 x 对应的最大利润为 $g(x)$ 。由第一步的结论我们可以得知，采样越多，方差越小，这意味着我们取到 p 比较大的差收益情况的概率大大减小（当次品率很大时，最大利润将会很小，对总体的期望利润会减小）。因此，我们可以使用 $g(x)$ 在 $x > p$ 处的期望来刻画收益，再减去采样的成本就可以得到我们的目标函数，即

$$h(n) = E_{x > p(n)}[g(x)] - n * cost_{\text{检测}} \quad (38)$$

由 1 中结论我们知道， p 的概率分布是由采样的次数 n 决定的，而检测成本也是由采样的次数决定的，因此整个目标函数都可以看做是采样次数的函数，我们记作 $h(n)$ 。对 $h(n)$ 进行定性的分析，当 n 很大时， n 对方差的影响微乎其微，反而对检测成本的影响较大，会导致检测成本快速上升。当 n 很小时，我们有较大概率取到 x 较大的部分，这会使得 $g(x)$ 的期望变小很多。因此，我们需要选择一个合适的 n 值，使得检测成本不

大的同时 $g(x)$ 有一个较好的期望。这与我们计算 n 的目的类似。因此从定性的角度看，我们使用这个目标函数可以很好地估计 n 的取值。

考虑第二问，我们将问题从一维上升至三维，此时我们的目标函数可以做以下的变化

$$h(n_1, n_2, n_3) = E_{x > p_1(n_1), y > p_2(n_2), z > p_3(n_3)}[g(x)] - (n_1 + n_2 + n_3) * cost_{\text{检测}} \quad (39)$$

记 $(n_1 + n_2 + n_3) * cost_{\text{检测}}$ 为 $fcost$, $x > p_1(n_1), y > p_2(n_2), z > p_3(n_3)$ 确定的空间为 ω , 则由期望的定义我们可以得到：

$$h(n_1, n_2, n_3) = \iiint_{\omega} g(x, y, z) f(x, y, z) dx dy dz - fcost \quad (40)$$

由于 x, y, z 相互独立，有

$$\begin{aligned} h(n_1, n_2, n_3) &= \iiint_{\omega} g(x, y, z) f(x, y, z) dx dy dz - fcost \\ &= \iiint_{\omega} g(x, y, z) f(x) f(y) f(z) dx dy dz - fcost \\ &\approx \sum_{x > p_1(n_1)} \sum_{y > p_2(n_2)} \sum_{z > p_3(n_3)} g(x, y, z) f(x) f(y) f(z) \end{aligned} \quad (41)$$

使用第二问的方法预处理出 $g(x, y, z)$ 的值，再枚举 n_1, n_2, n_3 ，求出 $f(x), f(y), f(z)$ ，枚举出最大的 $h(x, y, z)$ 所对应的 n_1, n_2, n_3 即为所求 **Step3**：

8.3 求解结果

如图 12，这张图反映了目标函数随着 n_1, n_2, n_3 取值的变化。由于采样点比较密集，不易观察到内部的情况，我们取了其中的一些点绘制成了图 13.0。当 $n_1=n_2=n_3=0.19*N$ 时取到最大值，平均利润为 29.17

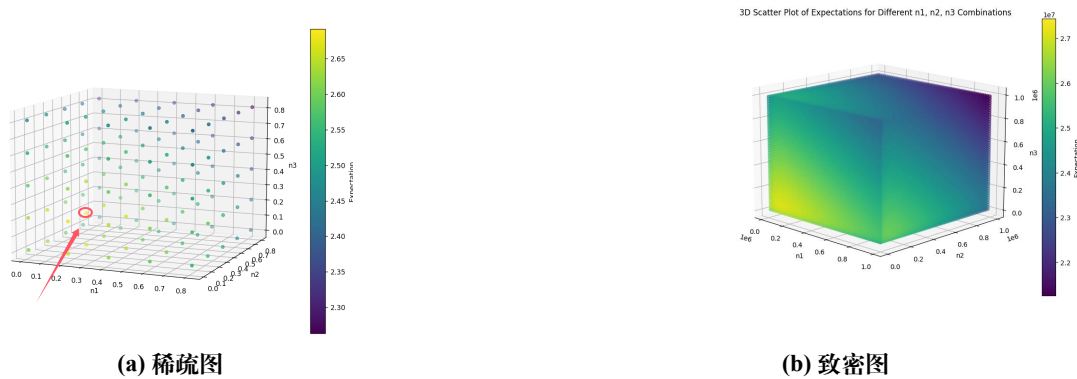


图 13

九、模型的分析与检验

9.1 不同总体 N 对抽样样本数的影响：

对于模型 1，我们通过修改模型的超参数 N 来检验我们模型的鲁棒性，结果如图 15:

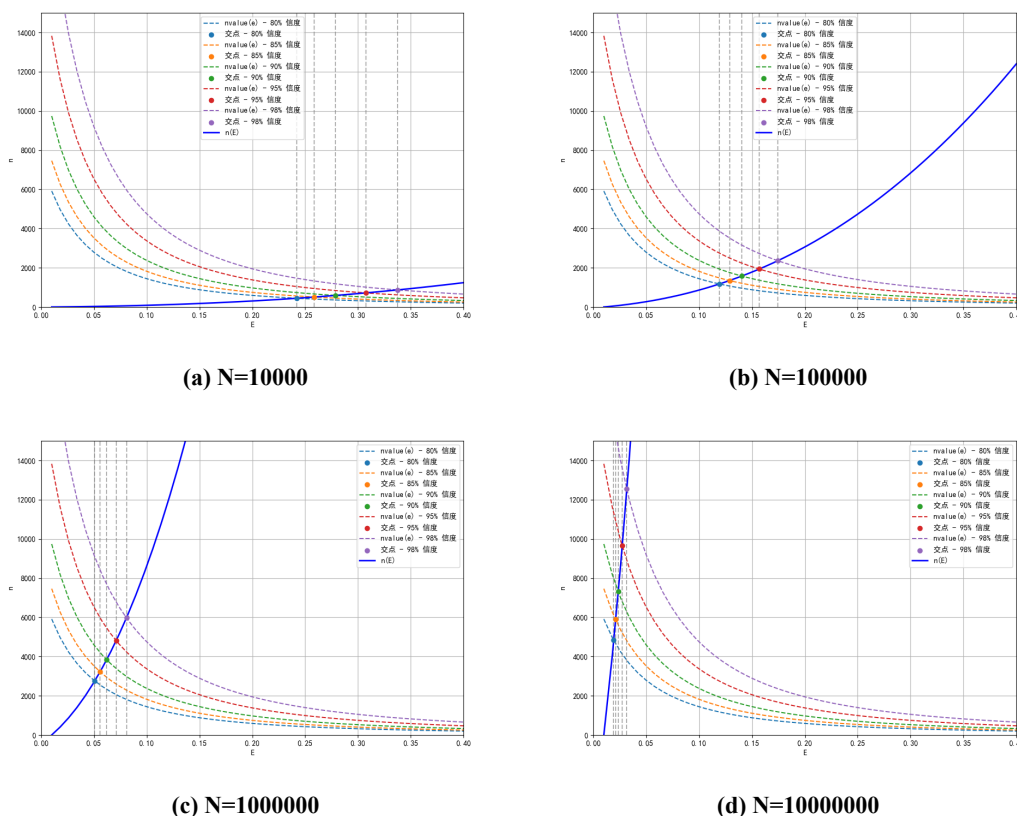


图 14 不同总体 N 对抽样样本数的影响

可以看到，随着 N 不断增长，误差成本会随之增大，说明我们需要投入更多的成本在检测上，这是符合常识与预期的，很好的证明了我们模型的鲁棒性。

9.2 不同零件采购数量对抽样样本数的影响：

对于模型 2 与模型 3，由于存在某种零件的价格远小于另一种零件（如问题 3 中的零配件 1 和零配件 3），所以在现实生活中，我们会多购买价格低的零件来避免价格高零件损耗带来的成本损失，于是在模型上改变了原始零件的数目，结果图 16:

可以看到，在不合理分配零件初始购买数量之后，最佳成本有所下降，证明了模型符合实际应用。

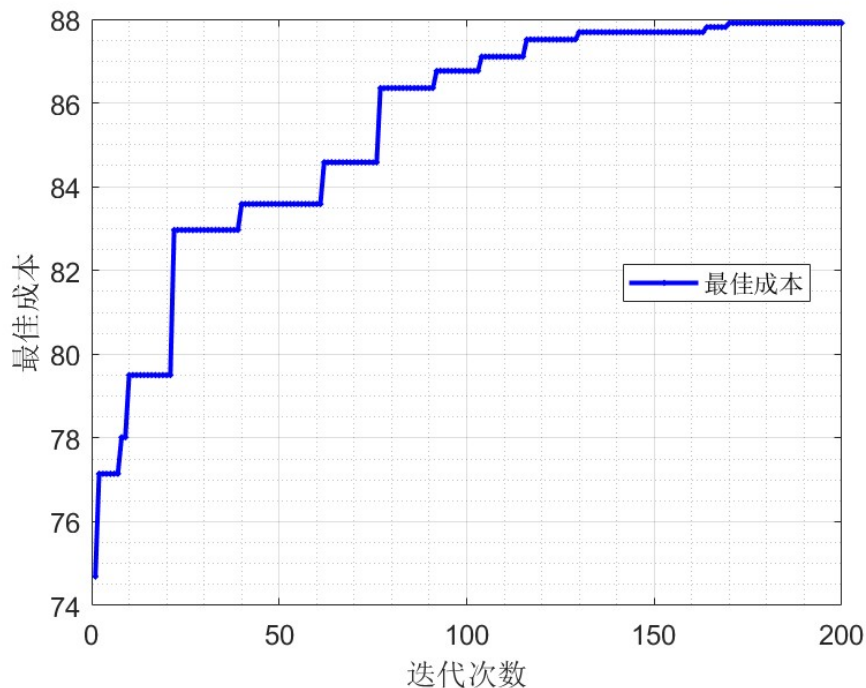


图 15 不合理初始分配

十、模型的评价

10.1 模型的优点

- 全文建模基于数学概率推导，并进行了模型对比，灵敏度分析，结果严谨全面。
- 模型二有较少的模型简化，所求解也为最优解，具有较强的现实应用意义。
- 遗传算法具有较快的搜索能力与较大的搜索域，改进克服了基因长度唯一的问题。

10.2 模型的缺点

- 问题三中的简化模型可能会丢失一些更优的选择，导致增加迭代次数。
- 问题四中对理想路径的采样不足可能导致误差。

10.3 模型的改进与推广

- 电子产品的制造具有一定周期，这就导致在生产过程中，成本、售价，零件状态都有可能发生变化。我们提出的三套模型分别适用于小规模简单工序生产，大规模复杂工序生产与不完全告知生产的情况，故可以将时间因素引入考量，提升模型鲁棒性。

参考文献

- [1] 林海凡. 李东波. 唐敦兵. 徐平. 模型及其遗传优化算法 [J][M]. 四川: 兵工自动化, 1999(03).
- [2] 赵进文. 复杂数据下经济建模与诊断研究[M]. 北京: 科学出版社, 2004.

附录 A 文件列表

文件名	功能描述
T1.figure.py	问题一画图代码 (图 2)
T1.equation.py	问题一程序代码
T2speed.py	问题二边界代码
B ₂ .cpp	问题二程序代码
T4sum.py	问题四程序代码

附录 B 代码

T1.figure.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import scipy.stats as stats
4
5 # 定义均值和标准差
6 mu = 0          # 均值
7 sigma = 1       # 标准差
8
9 # 生成正态分布的 x 值
10 x = np.linspace(mu - 4*sigma, mu + 4*sigma, 1000)
11
12 # 生成正态分布的概率密度函数 y 值
13 y = stats.norm.pdf(x, mu, sigma)
14
15 # 绘制正态分布曲线
16 plt.plot(x, y, color='blue')
17
18 # 计算 95% 单侧置信区间 (上侧)
19 upper_bound = stats.norm.ppf(0.95, loc=mu, scale=sigma)
20 #lower_bound =stats.norm.ppf()
21 # 填充置信区间区域 (单侧上限)
22 plt.fill_between(x, y, where=(x <= upper_bound), color='
    lightblue', alpha=0.5)
23
24 # 标注置信区间边界
25 plt.axvline(upper_bound, color='red')
```



```

26
27 # 添加图例、标题和标签
28 #plt.title('Normal Distribution with 95% One-sided Upper
    Confidence Interval')
29 plt.xlabel('X')
30 plt.legend()
31
32 # 显示图形
33 plt.show()

```

Tl.equation.py

```

1  import numpy as np
2  from scipy.integrate import solve_ivp
3  import matplotlib.pyplot as plt
4  C1 = 2
5  C2 = 4
6  N = 100000
7  p = 0.1
8  import pdb
9  def P(E):
10     if 1-p-E < 0:
11         print(1-p-E)
12         pdb.set_trace()
13     return (p + E) * np.log(p + E) + (1 - p - E) * np.log(1 -
        p - E)
14
15 # 定义微分方程  $dn/dE$  的表达式
16 def ode(E, n):
17     return (N * (E + p) * C2 + N * P(E)) / C1
18
19 # 设置 E 的取值范围
20 E_span = (0.01, 0.89) # E 的起始值和终止值
21 n_initial = [0.1] # n 的初始值
22
23 # 求解微分方程
24 sol = solve_ivp(ode, E_span, n_initial, method='RK45', t_eval=
    np.linspace(E_span[0], E_span[1], 100))
25 print(sol)
26 # 绘制结果，使用美化技巧
27 plt.figure(figsize=(10, 6))
28

```

```

29 # 设置线条的粗细、颜色、样式
30 plt.plot(sol.t, sol.y[0], label='n(E)', color='b', linestyle='
    -', linewidth=2, marker='o', markersize=5, markerfacecolor='
    red')
31
32 # 增加填充颜色
33 plt.fill_between(sol.t, sol.y[0], color='lightblue', alpha
    =0.3)
34
35 # 设置标签和标题
36 plt.xlabel('E', fontsize=14)
37 plt.ylabel('n', fontsize=14)
38
39 # 美化图例
40 plt.legend(fontsize=12, loc='best')
41
42 # 添加网格
43 plt.grid(True, linestyle='--', alpha=0.7)
44
45 # 显示图形
46 plt.tight_layout()
47 plt.show()

```

T2speed.py

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.interpolate import CubicSpline
4
5 # 定义常数
6 ccheck3 = 3
7 cexchange = 6
8 p3 = 0.1
9 n = 1
10 value = 56
11 break_cost = 5
12
13 z = np.zeros((101, 101))
14
15 for i in range(101):
16     for j in range(101):
17         prob1 = ((1 - i / 100) * (1 - j / 100) * (1 - p3)) *

```

```

    value * n - 6 * n - min(n * ccheck3, (1 - (1 - i / 100) * (1
    - j / 100) * p3) * n * cexchange) - break_cost * n
18     z[i, j] = prob1
19 i_vals, j_vals = np.meshgrid(np.arange(101), np.arange(101))
20 intersection_indices = np.where(np.isclose(z, 0, atol=1e-1))
21 if len(intersection_indices[0]) < 2:
22     raise ValueError("Not enough intersection points found to
    perform cubic spline fitting.")
23 i_points = i_vals[intersection_indices]
24 j_points = j_vals[intersection_indices]
25 z_points = z[intersection_indices]
26
27 sorted_indices = np.argsort(i_points)
28 i_points_sorted = i_points[sorted_indices]
29 j_points_sorted = j_points[sorted_indices]
30 i_points_unique, unique_indices = np.unique(i_points_sorted,
    return_index=True)
31 j_points_unique = j_points_sorted[unique_indices]
32 cubic_spline = CubicSpline(i_points_unique, j_points_unique)
33 i_fine = np.linspace(min(i_points_sorted), max(i_points_sorted
    ), 500)
34 j_fine = cubic_spline(i_fine)
35 plt.figure(figsize=(10, 6))
36 plt.scatter(i_points_sorted, j_points_sorted, color='red', s
    =80, edgecolor='black', zorder=5)
37 plt.plot(i_fine, j_fine, color='blue', linewidth=2, zorder=3)
38 plt.title('relationship between p1 and p2 in the case of bound
    ', fontsize=16, fontweight='bold')
39 plt.xlabel('i (p1)', fontsize=14)
40 plt.ylabel('j (p2)', fontsize=14)
41
42 plt.grid(True, linestyle='--', alpha=0.6)
43
44 plt.legend(fontsize=12)
45 plt.tight_layout()
46 plt.show()
47 print("Cubic spline fitting done.")

```

B_2.cpp

```

1 #include<bits/stdc++.h>
2 using namespace std;

```

```

3  #define ll long long
4  #define ld long double
5  struct LPJ{
6      ld num,cpl,gmdj,jccb,ifc=0; //个数,次品率,购买单价,检测成本
7  }; //零配件
8  LPJ P1,P2;
9  ld CP_cpl,CP_zpcb,CP_jccb,CP_scsj,DHSS,CJFY;
10 int num;
11 struct CP{
12     ld num,cpl,p1_num,p1_cpl,p2_num,p2_cpl; //个数,次品率,成品中p1的个数、次品率,成品中p2的个数、次品率
13 }; //成品
14 struct Path{
15     int step;
16     int choice;
17 };
18 Path path[100010],bestpath[100010];
19 ld bestans=0;
20 int h,besth;
21 ld besti,bestj;
22 void search(LPJ p1,LPJ p2,CP c,ld cb,ld lr,ll step);
23 void print(LPJ p1,LPJ p2,CP c,ld cb,ld lr)
24 {
25     cout<<"p1:"<<fixed<<setprecision(4)<<p1.num<<"    "<<p1.cpl<<
    <<"    "<<p1.gmdj<<"    "<<p1.jccb<<"\n";
26     cout<<"p2:"<<fixed<<setprecision(4)<<p2.num<<"    "<<p2.cpl<<
    <<"    "<<p2.gmdj<<"    "<<p2.jccb<<"\n";
27     cout<<"CP:"<<fixed<<setprecision(4)<<c.num<<"    "<<c.cpl<<
    <<"    "<<c.p1_cpl<<"    "<<c.p1_num<<"    "<<c.p2_cpl<<"    "<<c.
    p2_num<<"\n";
28 }
29 void step1_test_all(LPJ p1,LPJ p2,CP c,ld cb,ld lr)
30 {
31     cb=cb+p1.num*p1.jccb+p2.num*p2.jccb;
32     p1.num=p1.num*(1-p1.cpl);
33     p2.num=p2.num*(1-p2.cpl);
34     p1.ifc=1;
35     p2.ifc=1;
36     p1.cpl=p2.cpl=0;

```

```

37
38     search(p1,p2,c,cb,lr,2);
39 }
40 void step1_test_p1(LPJ p1,LPJ p2,CP c,ld cb,ld lr)
41 {
42     cb=cb+p1.num*p1.jccb;
43     p1.num=p1.num*(1-p1.cpl);
44     p1.ifc=1;
45     p1.cpl=0;
46
47     search(p1,p2,c,cb,lr,2);
48 }
49 void step1_test_p2(LPJ p1,LPJ p2,CP c,ld cb,ld lr)
50 {
51     cb=cb+p1.num*p2.jccb;
52     p2.num=p2.num*(1-p2.cpl);
53     p2.ifc=1;
54     p2.cpl=0;
55
56     search(p1,p2,c,cb,lr,2);
57 }
58 void step1_test_no(LPJ p1,LPJ p2,CP c,ld cb,ld lr)
59 {
60     search(p1,p2,c,cb,lr,2);
61 }
62 void step2_test1(LPJ p1,LPJ p2,CP c,ld cb,ld lr)
63 {
64     c.cpl=(c.cpl*c.num+min(p1.num,p2.num)*(1-(1-p1.cpl)*(1-p2.
65     cpl)*(1-CP_cpl)))/(c.num+min(p1.num,p2.num));
66     c.p1_cpl=(c.p1_cpl*c.p1_num+min(p1.num,p2.num)*p1.cpl)/(c.
67     p1_num+min(p1.num,p2.num));
68     c.p2_cpl=(c.p2_cpl*c.p2_num+min(p1.num,p2.num)*p2.cpl)/(c.
69     p2_num+min(p1.num,p2.num));
70     c.p1_num=c.p1_num+min(p1.num,p2.num);
71     c.p2_num=c.p2_num+min(p1.num,p2.num);
72     c.num=c.num+min(p1.num,p2.num);
73     ld pp=min(p1.num,p2.num);
74     p1.num=p1.num-pp;
75     p2.num=p2.num-pp;
76     lr=lr+CP_scsj*c.num;

```

```

74     search(p1,p2,c,cb,lr,4);
75 }
76 void step2_test2(LPJ p1,LPJ p2,CP c,ld cb,ld lr)
77 {
78     c.cpl=(c.cpl*c.num+min(p1.num,p2.num)*(1-(1-p1.cpl)*(1-p2.
cpl)*(1-CP_cpl)))/(c.num+min(p1.num,p2.num));
79     c.p1_cpl=(c.p1_cpl*c.p1_num)/(c.p1_num+min(p1.num,p2.num)
*(1-p1.cpl)*(1-p2.cpl));
80     c.p2_cpl=(c.p2_cpl*c.p2_num)/(c.p2_num+min(p1.num,p2.num)
*(1-p1.cpl)*(1-p2.cpl));
81     c.p1_num=c.p1_num+min(p1.num,p2.num)*(1-p1.cpl)*(1-p2.cpl)
;
82     c.p2_num=c.p2_num+min(p1.num,p2.num)*(1-p1.cpl)*(1-p2.cpl)
;
83     c.num=c.num+min(p1.num,p2.num);
84     ld pp=min(p1.num,p2.num)*(1-p1.cpl)*(1-p2.cpl);
85     p1.num=p1.num-pp;
86     p2.num=p2.num-pp;
87     lr=lr+CP_scsj*c.num*(1-c.cpl);
88     cb+=c.num*CP_jccb;
89     c.num=c.num*c.cpl;
90     c.cpl=1;
91     search(p1,p2,c,cb,lr,3);
92 }
93 void step3_test1(LPJ p1,LPJ p2,CP c,ld cb,ld lr)
94 {
95     // cout<<"step3_test1\n";
96     c.num=c.num*(1-c.cpl);
97     c.cpl=0;
98     c.p1_num=c.p2_num=c.num;
99     c.p1_cpl=c.p2_cpl=0;
100     if(lr-cb>bestans)
101     {
102         bestans=lr-cb;
103         besth=h;
104         besti=p1.cpl,bestj=p2.cpl;
105         for(int i=1;i<=besth;i++)
106         {
107             bestpath[i]=path[i];
108         }

```

```

109     }
110     if(p1.cpl!=0&& p2.cpl!=0)
111     {
112         cout<<p1.cpl<<" "<<p2.cpl<<endl;
113         num++;
114         // cout<<num;
115         cout<<"p end"<<p1.cpl<<" "<<p2.cpl<<endl;
116         cout<<"path:"<<endl;
117         for(int i=1;i<=h;i++)
118         {
119             cout<<"("<<path[i].step<<","<<path[i].choice<<") ";
120         }
121         cout<<"ans: "<<lr-cb<<endl;
122         //_sleep(100);
123     }
124     cb=lr=0;
125     return;
126 }
127 void step3_test2(LPJ p1,LPJ p2,CP c,ld cb,ld lr)
128 {
129     // print(p1,p2,c,cb,lr);
130     cb=cb+c.num*(c.cpl)*CJFY;
131     p1.num+=c.num*(1-c.cpl);
132     p2.num+=c.num*(1-c.cpl);
133     c.num=c.num*(1-c.cpl);
134     c.cpl=0;
135     c.p1_num=c.p2_num=c.num;
136     c.p1_cpl=c.p2_cpl=0;
137     // cout<<"ctmd"<<p1.num<<" "<<p2.num;
138     c.num=0;
139     search(p1,p2,c,cb,lr,1);
140 }
141 void step4_test(LPJ p1,LPJ p2,CP c,ld cb,ld lr)
142 {
143     cb=cb+c.num*c.cpl*(DHSS+CP_scsj);
144     c.num=c.num*c.cpl;
145     p1.num+=c.num;
146     p2.num+=c.num;
147     c.cpl=1;
148     c.p1_num=c.p2_num=c.num;

```

```

149
150     c.p1_cpl=c.p2_cpl=0;
151     search(p1,p2,c,cb,lr,3);
152 }
153 11 check(LPJ p1,LPJ p2,CP c,ld cb,ld lr)
154 {
155     // cout<<p1.ifc<<" "<<p2.ifc<<"\n";
156     ld x=(1-p1.cpl)*(1-CP_cpl)*CP_scsj-6-min(CP_jccb,(1-(1-p1.
157 ifc)*(1-CP_cpl))*DHSS)-CJFY;
158     ld y=(1-p2.cpl)*(1-CP_cpl)*CP_scsj-6-min(CP_jccb,(1-(1-p2.
159 ifc)*(1-CP_cpl))*DHSS)-CJFY;
160     ld z=(1-p1.cpl)*(1-p2.cpl)*(1-CP_cpl)*CP_scsj-6-min(
161 CP_jccb,(1-(1-p1.cpl)*(1-p2.cpl)*(1-CP_cpl))*DHSS)-CJFY;
162     if(p1.ifc&&p2.ifc) {
163         return 1;
164     }
165     else if(p2.ifc){
166         if((1-p1.cpl)*(1-CP_cpl)*CP_scsj-6-min(CP_jccb,(1-(1-
167 p1.ifc)*(1-CP_cpl))*DHSS)-CJFY>0) return 0;
168         else return 1;
169     }
170     else if(p1.ifc){
171         if((1-p2.cpl)*(1-CP_cpl)*CP_scsj-6-min(CP_jccb,(1-(1-
172 p2.ifc)*(1-CP_cpl))*DHSS)-CJFY>0) return 0;
173         else return 1;
174     }
175     else {
176         if((1-p1.cpl)*(1-p2.cpl)*(1-CP_cpl)*CP_scsj-6-min(
177 CP_jccb,(1-(1-p1.cpl)*(1-p2.cpl)*(1-CP_cpl))*DHSS)-CJFY>0)
178 return 0;
179         else return 1;
180     }
181     return 0;
182 }
183 void search(LPJ p1,LPJ p2,CP c,ld cb,ld lr,11 step)
184 {
185     // cout<<step<<"\n";
186     ld ccbb=cb,11rr=lr;
187     //print(p1,p2,c,cb,lr);

```



```

182     if(step==1)
183     {
184         if(!p1.ifc&&!p2.ifc)
185         {
186             path[++h].step=1;
187             path[h].choice=1;
188             step1_test_all(p1,p2,c,cb,lr);
189             h--;
190         }
191         if(!p1.ifc)
192         {
193             path[++h].step=1;
194             path[h].choice=2;
195             step1_test_p1(p1,p2,c,cb,lr);
196             h--;
197         }
198         if(!p2.ifc)
199         {
200             path[++h].step=1;
201             path[h].choice=3;
202             step1_test_p2(p1,p2,c,cb,lr);
203             h--;
204         }
205         path[++h].step=1;
206         path[h].choice=4;
207         step1_test_no(p1,p2,c,cb,lr);
208         h--;
209     }
210     else if(step==2)
211     {
212         path[++h].step=2;
213         path[h].choice=1;
214         step2_test1(p1,p2,c,cb,lr); //不检测
215         h--;
216         path[++h].step=2;
217         path[h].choice=2;
218         step2_test2(p1,p2,c,cb,lr); //检测
219         h--;
220     }
221     else if(step==3)

```

```

222     {
223         if(p1.ifc==1&&p2.ifc==1)
224         {
225             if(CP_scsj*(1-CP_cpl)>CP_zpcb+CJFY)
226             {
227                 cb+=c.num*c.cpl*(CP_zpcb+(CP_zpcb+CJFY)*CP_cpl
/((1-CP_cpl)*(1-CP_cpl)));
228                 lr+=c.cpl*c.num*CP_scsj;
229             }
230             p1.num=0,p2.num=0;
231
232         }
233         if(check(p1,p2,c,cb,lr)||h>=50)
234         {
235             path[++h].step=3;
236             path[h].choice=1;
237             step3_test1(p1,p2,c,cb,lr); //不拆解
238             h--;
239         }
240         else {
241             path[++h].step=3;
242             path[h].choice=2;
243             step3_test2(p1,p2,c,cb,lr); //拆解
244             h--;
245         }
246     }
247     else if(step==4)
248     {
249         path[++h].step=4;
250         path[h].choice=1;
251         step4_test(p1,p2,c,cb,lr);
252         h--;
253     }
254 }
255 int main()
256 {
257
258     freopen("out.txt","w",stdout);
259     cin>>P1.cpl>>P1.gmdj>>P1.jccb>>P2.cpl>>P2.gmdj>>P2.jccb>>
CP_cpl>>CP_zpcb>>CP_jccb>>CP_scsj>>DHSS>>CJFY;

```

```

260     CP c;
261     P1.num=1;
262     P2.num=1;
263     c.num=c.cpl=c.p1_cpl=c.p1_num=c.p2_cpl=c.p2_num=0;
264     search(P1,P2,c,P1.num*P1.gmdj+P2.num*P2.gmdj,0,1);
265     cout<<"path:"<<endl;
266     for(int i=1;i<=besth;i++)
267     {
268         cout<<bestpath[i].step<<" "<<bestpath[i].choice<<" ";
269     }
270     cout<<"ans: "<<bestans<<endl;
271     cout<<"besth"<<besth<<endl;
272     // cout<<"besti,j"<<besti<<" "<<bestj;
273     return 0;
274 }

```

T4sum.py

```

1  import numpy as np
2  import math
3  import os
4  import time
5  from tqdm import *
6  import matplotlib.pyplot as plt
7  from mpl_toolkits.mplot3d import Axes3D
8  costn1=2
9  costn2=3
10 costn3=3
11 # 定义概率密度函数 f(x)
12 def distribution_function(x, p, Z, N):
13     E = Z * math.sqrt((p * (1 - p)) / N)
14     variance = 2 * math.log(2) * E
15     coeff = 1 / math.sqrt(2 * math.pi * variance)
16     exponent = -((x - p) ** 2) / (2 * variance)
17     return coeff * math.exp(exponent)
18
19 # 读取 g(x, y, z) 的值从 txt 文件中，并转换为三维数组
20 def read_g_values(filename, x_len, y_len, z_len):
21     with open(filename, 'r') as file:
22         values = list(map(float, file.read().split()))
23         expected_size = x_len * y_len * z_len
24         if len(values) != expected_size:

```

```

25         raise ValueError(f"g_values size ({len(values)}) does
not match expected size ({expected_size}).")
26     return np.reshape(values, (x_len, y_len, z_len))
27
28 # 定义求期望的函数
29 def calculate_expectation(p, Z, n1, n2, n3, g_values, x_values
, y_values, z_values):
30     expectation = 0
31     total_weight = 0
32     for i, x in enumerate(x_values):
33         for j, y in enumerate(y_values):
34             for k, z in enumerate(z_values):
35                 f_x = distribution_function(x, 0.1, Z, n1)
36                 f_y = distribution_function(y, 0.1, Z, n2)
37                 f_z = distribution_function(z, 0.1, Z, n3)
38                 g_xyz = g_values[i][j][k]
39                 weight = f_x * f_y * f_z
40                 expectation += g_xyz * weight
41                 total_weight += weight
42     return expectation / total_weight if total_weight != 0
else 0
43
44 # 找到使期望最大的 n1, n2, n3 并绘制三维散点图
45 def find_best_n_and_plot_3d(p, Z, g_values, x_values, y_values
, z_values):
46     best_n1, best_n2, best_n3 = None, None, None
47     max_expectation = -float('inf')
48     n1_list = []
49     n2_list = []
50     n3_list = []
51     expectations = []
52
53     # 分别枚举 n1, n2, n3, 从1到100, 步长10 (减少范围)
54     for n1 in tqdm(range(10000, 1000000, 20000)):
55         for n2 in range(10000, 1000000, 20000):
56             for n3 in range(10000, 1000000, 20000):
57                 expectation = calculate_expectation(p, Z, n1,
n2, n3, g_values, x_values, y_values, z_values)
58                 n1_list.append(n1/1000000)
59                 n2_list.append(n2/1000000)

```

```

60         n3_list.append(n3/1000000)
61         expectations.append(expectation*1000000-n1*
costn1-n2*costn2-n3*costn3)
62
63         if expectation > max_expectation:
64             max_expectation = expectation
65             best_n1, best_n2, best_n3 = n1, n2, n3
66
67     # 绘制三维散点图
68     fig = plt.figure(figsize=(10, 8))
69     ax = fig.add_subplot(111, projection='3d')
70
71     # 将 n1, n2, n3 和期望值转换为 numpy 数组
72     n1_arr = np.array(n1_list)
73     n2_arr = np.array(n2_list)
74     n3_arr = np.array(n3_list)
75     expectations_arr = np.array(expectations)
76
77     # 使用散点图绘制 n1, n2, n3 的期望值
78     scatter = ax.scatter(n1_arr, n2_arr, n3_arr, c=
expectations_arr, cmap='viridis', marker='o')
79     ax.set_xlabel('n1')
80     ax.set_ylabel('n2')
81     ax.set_zlabel('n3')
82     plt.title('3D Scatter Plot of Expectations for Different
n1, n2, n3 Combinations')
83
84     # 添加颜色条
85     cbar = plt.colorbar(scatter)
86     cbar.set_label('Expectation')
87
88     plt.show()
89
90     return best_n1, best_n2, best_n3, max_expectation
91
92 # 示例参数
93 p = 0.1
94 Z = 1.959963984540054
95
96 # 根据 g_values 的大小 (19 x 19 x 19) 定义 x, y, z 的取值范围

```

```

97 x_values = np.linspace(0.1, 0.66, 19) # 19 个取值
98 y_values = np.linspace(0.1, 0.66, 19) # 19 个取值
99 z_values = np.linspace(0.1, 0.66, 19) # 19 个取值
100
101 # 读取 g(x, y, z) 的值, 确保与 x, y, z 的长度匹配
102 current_dir = os.path.dirname(os.path.abspath(__file__))
103 filename = os.path.join(current_dir, 'g_values.txt') # 使用
    os.path.join 来处理路径
104
105 # 读取 g_values 并将其转换为三维数组
106 g_values = read_g_values(filename, len(x_values), len(y_values
    ), len(z_values))
107
108 # 找到使期望最大的 n1, n2, n3 并绘制三维散点图
109 best_n1, best_n2, best_n3, max_expectation =
    find_best_n_and_plot_3d(p, Z, g_values, x_values, y_values,
    z_values)
110 print(best_n1, best_n2, best_n3, max_expectation)
111 print(f"The best n1 = {best_n1}, n2 = {best_n2}, n3 = {best_n3
    } with an expectation of {max_expectation}")

```